

Apache HBase

Fast Access to Big Data

Distributed · NoSQL · Wide-Column · Versioned

HDFS

Hive

Spark

HBase

Where Are We?

Chaque composant répond à une responsabilité différente

HDFS

Storage

YARN

Resources

Spark

Processing

Hive

SQL Analytics

HBase

Fast Key-Based
Access

Aujourd'hui : comprendre pourquoi HBase complète HDFS, Spark et Hive - sans les remplacer.

We Already Have HDFS, Spark and Hive

Pourquoi ajouter une autre technologie ?

HDFS

Excellent pour de gros fichiers et les archives.

Spark

Excellent pour transformer et agréger à grande échelle.

Hive

Excellent pour les requêtes SQL analytiques.

INC-000154 ?

Et si je connais déjà l'identifiant exact de l'incident ?

Je veux son statut, son propriétaire et son chemin de preuve immédiatement - sans scanner des années de logs.

HBase répond à un besoin d'accès opérationnel ciblé, différent du stockage massif et de l'analytics.

The Operational Access Problem

Retrouver rapidement une information connue en partie

INC-000154

GET by RowKey

status

INVESTIGATING

application

payment-api

owner

team_payments

evidence

/audit/...

Cas typique HBase : retrouver une ligne ou une petite plage de lignes à partir d'une clé.

Why Not HDFS?

HDFS optimise le débit sur de gros fichiers : pas l'accès ligne par ligne

HDFS - excellent pour

- gros fichiers
- lectures séquentielles
- haut débit
- archives immuables

Moins naturel pour

- GET une ligne
- UPDATE un état
- recherche par clé
- lecture aléatoire ciblée

HBase
key-based access

HDFS reste la source de vérité pour les fichiers ; HBase fournit une couche d'accès plus fine.

Why Not Hive?

Une requête SQL peut exprimer le besoin mais le workload reste analytique

HiveQL

```
SELECT *  
FROM incidents  
WHERE incident_id = 'INC-000154';
```

analytics engine

Batch / SQL Analytics

Pour un lookup opérationnel très ciblé, HBase est plus naturel si la RowKey est bien conçue.

Hive et HBase peuvent exposer les mêmes données métier sous des modèles d'accès différents.

HBase in One Sentence

HBase provides distributed, key-based read/write access to very large datasets.

DISTRIBUTED

KEY-BASED

READ / WRITE

VERSIONED

HBase est une base NoSQL distribuée orientée colonnes, inspirée du modèle Bigtable.

What Can HBase Do?

Six capacités à retenir

PUT

Écrire ou mettre à jour

GET

Lire une RowKey connue

SCAN

Lire une plage ordonnée

FILTER

Limiter les résultats

VERSIONS

Conserver plusieurs états

SCALE OUT

Distribuer les régions

Le TP utilisera directement put, get, scan, filters, versions et conception de RowKey.

HBase Is NoSQL

Le modèle n'est pas conçu autour des jointures relationnelles

No JOIN-first design

Les accès principaux doivent être anticipés.

No foreign keys

La cohérence relationnelle n'est pas le modèle central.

Sparse rows

Toutes les lignes n'ont pas les mêmes qualifieurs.

Access-pattern driven

La RowKey découle des requêtes principales.

En HBase, on modélise d'abord les accès ; la structure suit ensuite.

Wide-Column Model

Une ligne peut regrouper plusieurs familles et qualifieurs

RowKey

payment-api#20260115#...

event

level
status_code
response_time_ms

tech

source_team
hdfs_path

audit

ingestion_status
reason

Les familles regroupent des colonnes stockées ensemble ; les qualifieurs restent flexibles.

Le modèle de données HBase

RowKey, familles, qualifiers, cellules et versions



Anatomy of an HBase Cell

Une cellule est identifiée par quatre dimensions

RowKey

payment-
api#20260115#9999
999988#req-002

Family

event

Qualifier

level

Timestamp

173693...

Value

WARN

RowKey + famille + qualifieur + timestamp identifient une version précise de la cellule.

RowKey

The primary access path

payment-api#20260115#9999999988#req-002

application

payment-api

date

20260115

reverse time

9999999988

request

req-002

Une bonne RowKey encode les dimensions utiles aux principaux patterns d'accès.

Rows Are Sorted by RowKey

L'ordre lexicographique rend les range scans possibles

```
auth-service#20260115#...
```

```
auth-service#20260116#...
```

```
payment-api#20260115#...
```

```
payment-api#20260115#...
```

```
payment-api#20260116#...
```

Mettre `app_id` puis `date` en préfixe permet de regrouper les événements d'une application/journée.

Column Families

Les familles sont structurantes et définies à la création de la table

event

Données fonctionnelles de l'événement

tech

Métadonnées techniques de collecte

audit

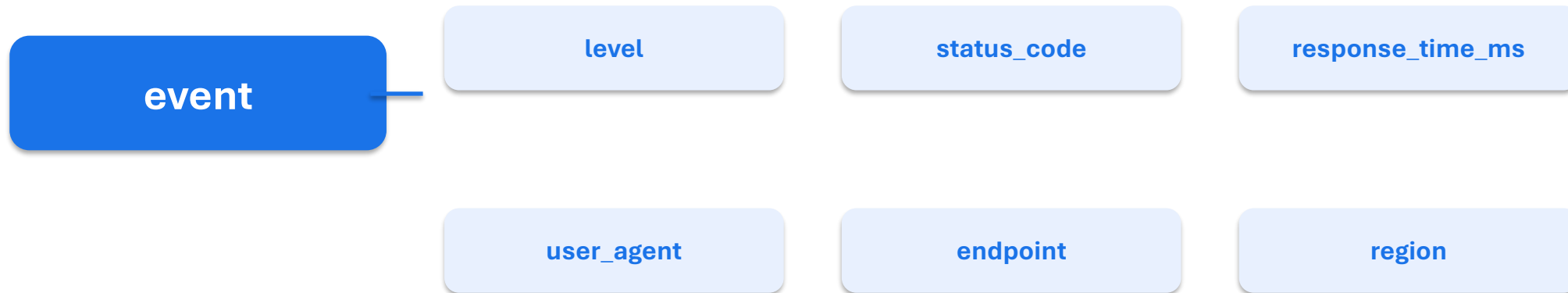
Informations de preuve et de contrôle

Éviter une famille par champ : les familles ont un impact physique sur le stockage et les lectures.

Le TP crée précisément les familles event, tech et audit.

Qualifiers Are Flexible

Les qualifiers peuvent évoluer sans redéfinir toute la table



Familles = structure stable ; qualifiers = flexibilité à l'intérieur de chaque famille.

Sparse Data Model

Deux lignes peuvent avoir des colonnes différentes sans stocker des NULL partout

Row A

event:level

event:status_code

tech:source_team

Row B

event:level

audit:reason

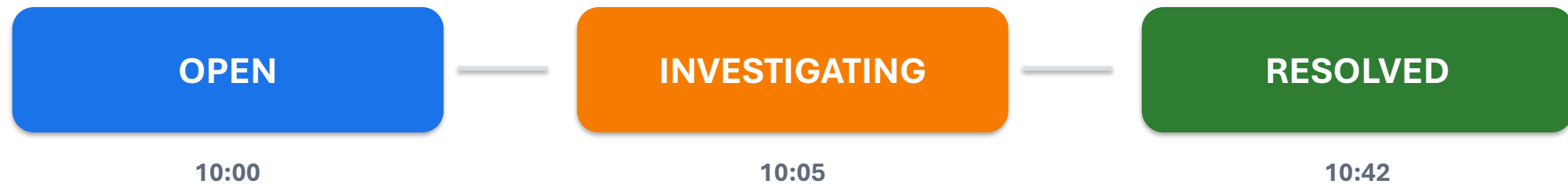
audit:ingestion_status

HBase stocke seulement les cellules présentes : le modèle est naturellement sparse.

Versioned Cells

Une même cellule peut conserver plusieurs états

state:status



Le nombre de versions est configuré par famille. Le versionnement technique n'est pas forcément un historique métier complet.

Le TP utilise plusieurs versions pour suivre le cycle de vie d'un incident.

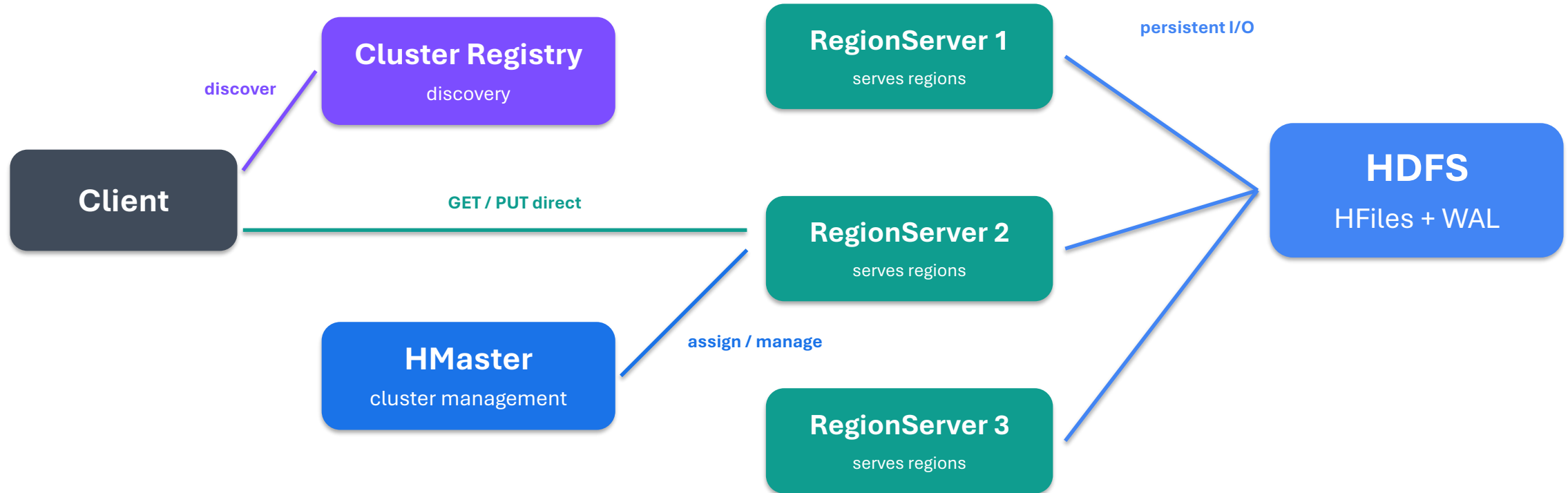
Comment HBase fonctionne

Masters, RegionServers, régions et stockage HDFS



HBase Architecture

Des composants de contrôle séparés du chemin principal des données



Le client découvre la région puis parle directement au RegionServer ; le HMaster administre le cluster ; HDFS assure la persistance.

HMaster

Le control plane de HBase

Table operations

Create, alter, disable, delete

Region assignment

Affecter / réaffecter les régions

Cluster balancing

Répartir la charge entre RegionServers

Recovery coordination

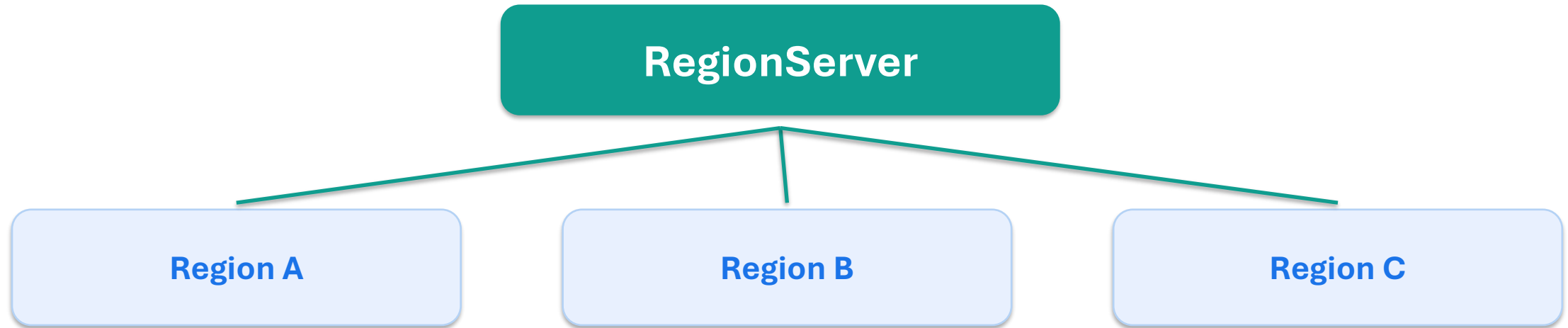
Réagir à certains échecs serveur

Point clé : le HMaster n'est pas sur le chemin normal de chaque GET ou PUT.

Le data plane principal passe entre le client et les RegionServers.

RegionServer

The data-serving process



Un RegionServer héberge plusieurs régions et répond aux lectures/écritures des lignes qu'elles contiennent.

What Is a Region?

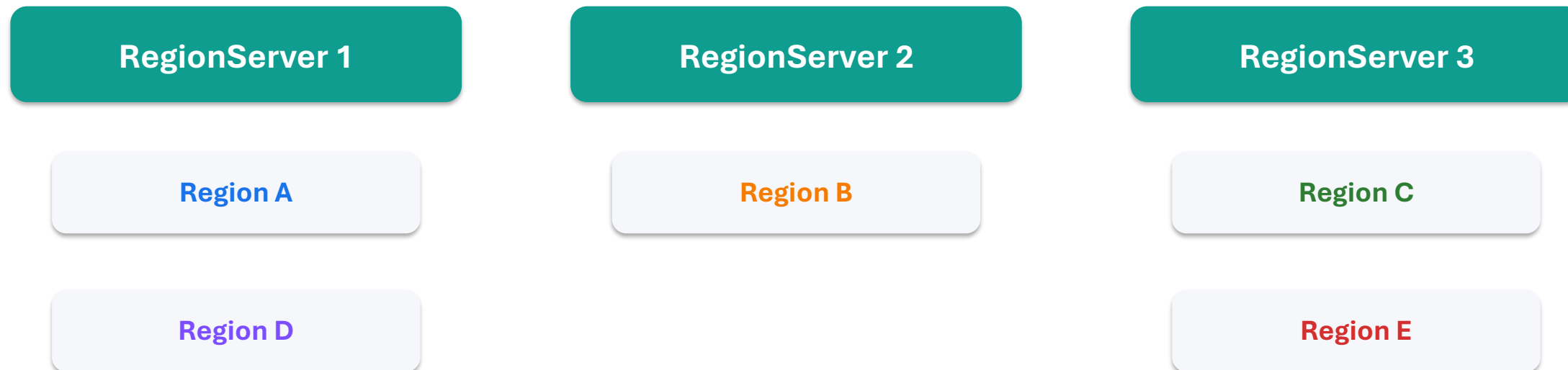
Une table est découpée en plages contiguës de RowKeys



Quand une région grossit, elle peut être scindée pour permettre le scale-out.

Regions Across RegionServers

Le scale-out vient de la distribution des régions



Ajouter des RegionServers augmente la capacité de service lorsque les régions peuvent être réparties.

Why RowKey Controls Distribution

Les RowKeys proches sont stockées dans les mêmes régions



La RowKey influence directement la distribution des données et la répartition de la charge.

How a Client Finds Data

Découvrir la région, puis contacter directement le RegionServer



Après découverte, le client met en cache les localisations pour éviter de refaire la résolution à chaque accès.

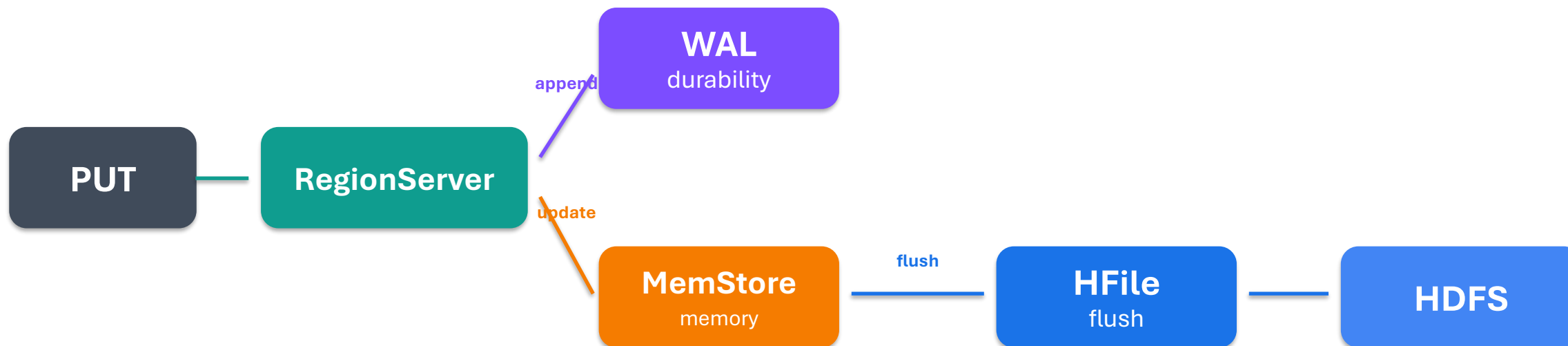
Write Path & Read Path

Ce qui se passe réellement lors d'un PUT ou d'un GET



HBase Write Path

Durabilité puis écriture en mémoire avant persistance en HFiles



Le WAL protège la durabilité ; le MemStore absorbe les écritures ; les flushes produisent des HFiles sur HDFS.

WAL - Write Ahead Log

Enregistrer le changement avant de dépendre uniquement de la mémoire



Durability

Le journal permet de rejouer des écritures après un crash.

Recovery

Le cluster peut reconstruire un état récent à partir du WAL.

Le WAL est un journal de durabilité, pas la structure finale de stockage des données.

MemStore

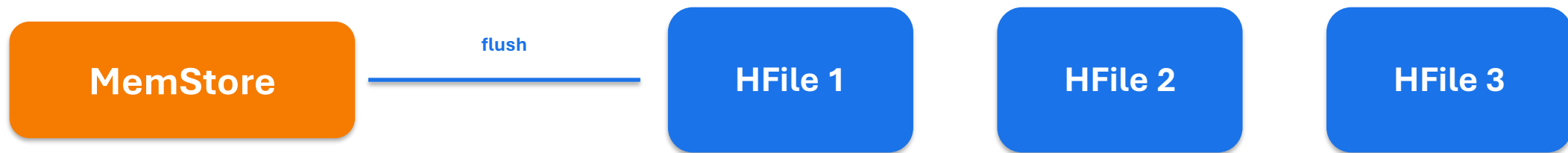
Accumuler les écritures en mémoire avant le flush



Le MemStore améliore l'efficacité des écritures en regroupant les modifications avant persistance.

HFiles

Le format persistant des données HBase sur HDFS



Avec le temps, plusieurs HFiles peuvent s'accumuler pour une même Store / famille de colonnes.

HFiles sont immuables ; les nouvelles écritures créent de nouveaux fichiers plutôt que de modifier les anciens.

Why Compaction Exists

Réduire le nombre de fichiers et réorganiser les données persistées

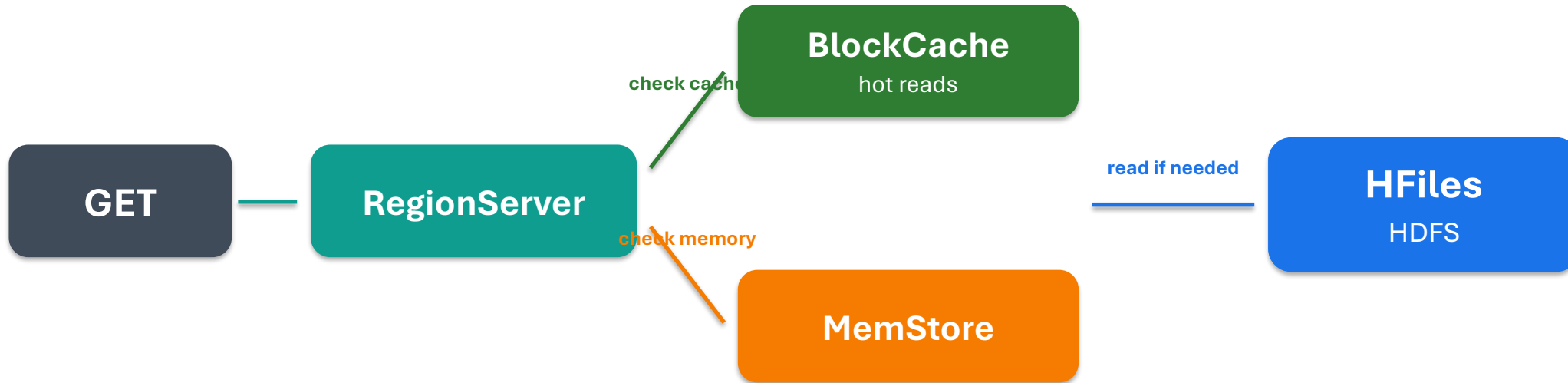


Compaction réduit la fragmentation et facilite certaines lectures, mais consomme CPU, disque et I/O : elle fait partie du coût opérationnel HBase.

HBase échange des écritures rapides contre un travail de maintenance asynchrone en arrière-plan.

HBase Read Path

Lire d'abord les données les plus proches, puis les fichiers persistés si nécessaire



Le détail exact dépend des caches et structures internes, mais le client n'effectue pas un scan complet de la table.

GET vs SCAN

Deux modes d'accès complémentaires

GET

Je connais la RowKey exacte.

- une ligne ciblée
- très efficace si la clé est connue

SCAN

Je connais un intervalle de RowKeys.

- plusieurs lignes contiguës
- efficace si le préfixe de clé correspond au besoin

Le modèle de RowKey doit permettre d'utiliser GET ou des scans bornés plutôt que des scans complets.

La RowKey est la décision centrale

Concevoir les clés à partir des patterns d'accès



Your Schema Starts With the Query

En HBase, le modèle découle des accès principaux

1

Quelle clé connaît l'utilisateur ?

2

Une ligne ou une plage ?

3

Quelle fraîcheur ?

4

Combien de versions ?

5

Quelles données restent dans HDFS ?

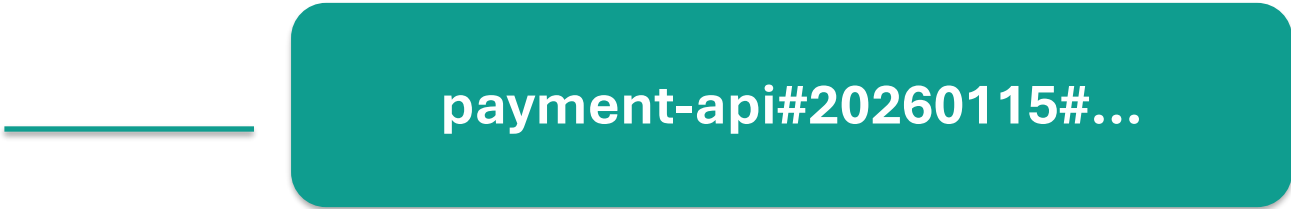
Un bon modèle HBase commence par les requêtes, pas par la liste de colonnes.

Access Pattern #1

Tous les événements d'une application pour une journée

Need

payment-api
2026-01-15



payment-api#20260115#...

Les lignes pertinentes deviennent contiguës dans l'ordre des RowKeys.

Préfixer par app_id puis date optimise le scan de cette application sur cette journée.

Range Scan

STARTROW et STOPROW bornent précisément la plage à lire

HBase Shell

```
STARTROW => 'payment-api#20260115#'  
STOPROW  => 'payment-api#20260116#'
```

payment-api#20260114#...

payment-api#20260115#001

payment-api#20260115#002

payment-api#20260116#...

La RowKey choisie transforme une requête métier en un scan borné et efficace.

Access Pattern #2

Retrouver un événement uniquement par request_id



Problème : avec request_id seul, on ne connaît pas le début de la RowKey. Un GET direct n'est donc pas possible.

Un seul modèle de RowKey n'optimise pas toutes les requêtes possibles.

Multiple Access Patterns → Multiple Tables?

La duplication contrôlée peut être plus efficace qu'un accès générique lent

application_events

app#date#...

Optimisée pour les scans par application
et date.

events_by_request

request_id#...

Optimisée pour les GET par identifiant de
requête.

En NoSQL, plusieurs vues matérialisées ou tables d'index peuvent servir plusieurs patterns d'accès.

The Hotspot Problem

Une mauvaise RowKey peut concentrer toutes les écritures sur une seule région



La distribution lexicographique des RowKeys signifie que les préfixes monotones demandent une attention particulière.

Salting

Distribuer les écritures en ajoutant un préfixe artificiel

salt00

salt00#payment-api#20260115#req-001

salt01

salt01#payment-api#20260115#req-002

salt02

salt02#payment-api#20260115#req-003

salt03

salt03#payment-api#20260115#req-004

Benefit

Écritures réparties sur davantage de régions.

Trade-off

Un scan logique peut nécessiter plusieurs préfixes / scans.

Le salage résout un problème de distribution au prix d'une complexité de lecture supplémentaire.

Reverse Timestamp

Conserver les événements récents près du début d'un préfixe logique

`app_id#date#reverse_ts#request_id`

$\text{reverse_ts} = \text{MAX_TS} - \text{timestamp}$

08:00

9999999999

08:05

9999999994

08:10

9999999989

Le reverse timestamp est utile dans certains patterns, mais doit être validé avec l'ordre exact recherché.

HBase dans l'écosystème

Choisir le bon outil selon le type d'accès



HBase ≠ HDFS

Deux abstractions de stockage très différentes

Dimension	HDFS	HBase
Unit	Files / blocks	Rows / cells
Access	Large scans	GET / range scan
Strength	Archive / throughput	Targeted access
Project role	Source of truth	Index / operational state

HBase repose sur HDFS mais n'expose pas le même modèle d'accès aux applications.

HBase ≠ Hive

SQL analytique vs accès ciblé par clé

Hive

- SQL analytics
- Scans / agrégations
- Data Warehouse
- Historique global

HBase

- GET / range scans
- Key-based access
- Operational lookup
- État courant / versions

Les deux peuvent coexister : Hive pour analyser, HBase pour servir des accès rapides spécifiques.

HBase ≠ Spark

Store / Serve versus Compute / Transform

Spark

compute · transform ·
aggregate

HBase

store · index · serve

Spark peut calculer un indicateur ; HBase peut ensuite servir rapidement cet indicateur à une application opérationnelle.

Ne pas confondre moteur de calcul distribué et base de données distribuée.

Choose the Right Tool

Le besoin dicte la technologie

Conserver 5 ans de logs

HDFS

Calculer les indicateurs

Spark

SQL annuel / reporting

Hive

Trouver INC-0001

HBase

Une architecture mature assemble plusieurs composants au lieu de forcer un seul outil à tout faire.

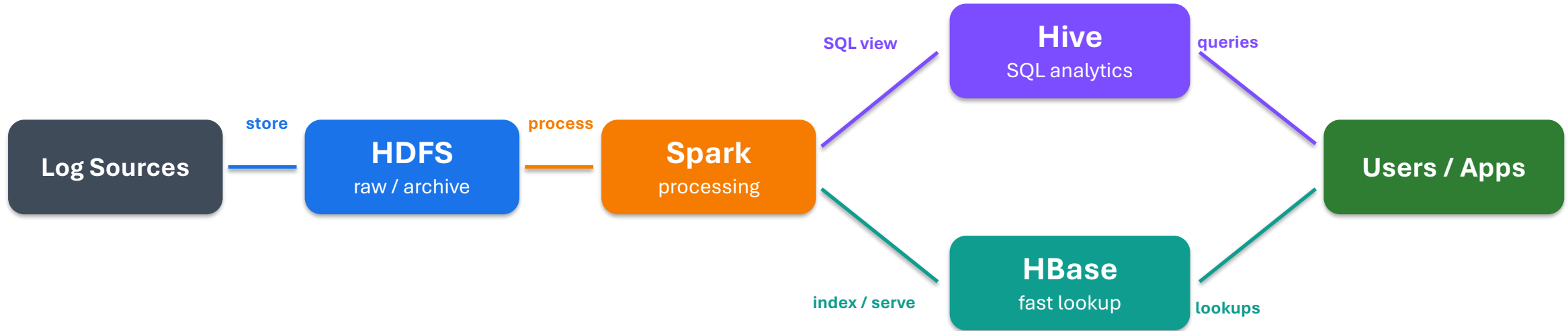
Projet fil rouge DORA

HBase comme couche d'accès rapide et d'indexation



Back to the DORA Platform

Les preuves restent dans HDFS ; HBase sert les accès ciblés



HDFS = source de vérité ; Spark = transformations ; Hive = analyse SQL ; HBase = accès opérationnels ciblés.

Use Case 1 - Application Events

Indexer les événements utiles sans déplacer la preuve complète

application_events

RowKey

app_id#date#reverse_timestamp#request_id

event

level · status · latency

tech

source_team · hdfs_path

audit

ingestion_status · proof metadata

Le chemin HDFS relie l'index HBase à la preuve complète stockée dans le Data Lake.

Use Case 2 - Incident Tracking

Accéder rapidement à un incident et conserver plusieurs états

incidents

RowKey

incident#20260115#INC-0001

identity

app_id · criticality

state

status · owner_team

links

first_event · HDFS evidence

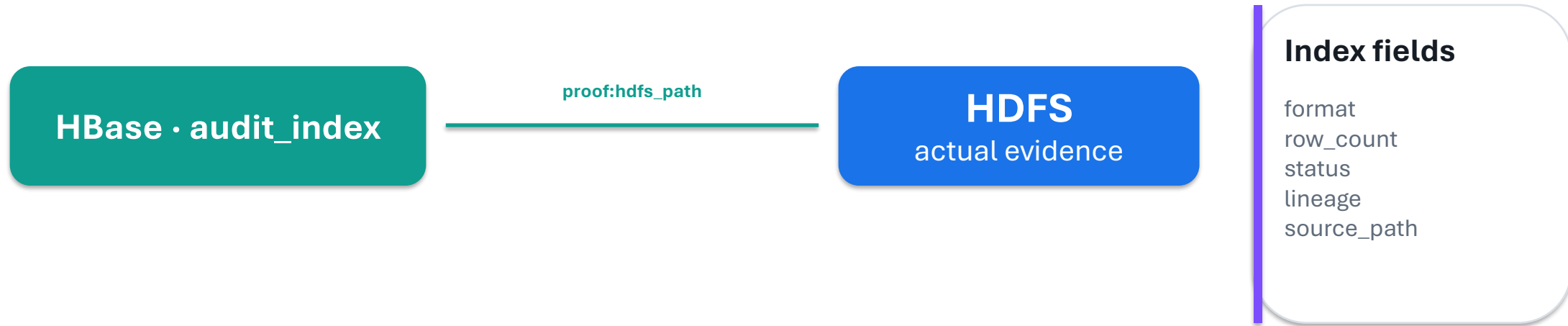
audit

reason · versions

Les familles state et audit peuvent garder plusieurs versions pour suivre l'évolution de l'incident.

Use Case 3 - Audit Index

HBase indexe la preuve ; HDFS conserve le contenu



Ce modèle accélère la localisation des preuves sans dupliquer les archives complètes dans HBase.

Why Not Store the Evidence in HBase?

Utiliser chaque technologie pour ce qu'elle fait le mieux

HBase

- metadata
- index
- status
- selected fields
- fast lookups

HDFS

- large immutable evidence
- raw logs
- archives
- full files
- long-term retention

La duplication de gros contenus dans HBase augmenterait le coût sans améliorer le rôle d'archive du Data Lake.

One Incident, Multiple Versions

Versionner l'état sans perdre les transitions récentes



TP

```
get '...:incidents', 'incident#...#INC-0001', { COLUMN => 'state:status', VERSIONS => 5 }
```

Limiter le nombre de versions évite une croissance incontrôlée et doit être aligné avec le besoin d'audit.

Final Architecture Responsibilities

Une responsabilité claire par composant

HDFS

Source de vérité / archives

YARN

Ressources

Spark

Transformations / indicateurs

Hive

SQL / Data Warehouse

HBase

Index · état · lookup rapide

Cette séparation des responsabilités doit apparaître dans l'architecture finale du projet.

HBase Is Not the Data Lake

**HBase accelerates access to selected information.
It does not replace the Cold Storage Data Lake.**

HBase

fast access / index / state

HDFS

evidence / archive / source of truth

Le projet garde les logs bruts et preuves complètes dans HDFS, même si HBase en indexe certaines métadonnées.

TP07 - HBase

Six missions pour passer de la théorie au modèle DORA

1 · DISCOVER

hbase shell · status · namespace

2 · MODEL

RowKey · families · qualifiers

3 · OPERATE

put · get · scan · filters

4 · DESIGN

range scans · hotspots · salting

5 · VERSION

incident lifecycle · versions

6 · INTEGRATE

HBase ↔ HDFS · audit index · DORA

Objectif : justifier où HBase apporte de la valeur et où il faut continuer à utiliser HDFS, Spark ou Hive.

Session 07 - Key Takeaways

Ce qu'il faut savoir défendre à la soutenance

1

**HBase = accès
distribué par clé**

2

**RowKey = choix
d'architecture**

3

**Regions = unité de
distribution**

4

**Families + versions =
modèle physique**

5

**HDFS reste la source
de vérité**

6

**HBase sert le projet via
index / incidents**

Prochaine étape : consolider l'architecture, les choix techniques, les limites et les perspectives du projet final.

Sources principales

Références utilisées pour cette session

Cours / TP

github.com/elomedah/big-data · [syllabus.md](#) · [tp/07-hbase/README.md](#)

Apache HBase

Apache HBase Reference Guide · hbase.apache.org/book.html

Modèle TP

[application_events](#) · [incidents](#) · [audit_index](#) · [RowKey](#) / [versions](#) / [scans](#)

Les exemples DORA sont pédagogiques et servent à justifier les responsabilités des composants dans l'architecture du module.